

DOCUMENTATION TECHNIQUE

Lambda : archive_anaplan-prd

Archivage mensuel des exports Anaplan — Plateforme ACF

Périmètre	BPIFrance — Plateforme ACF / Pipeline Anaplan
Environnement	Production (prd)
Runtime	Python 3.x — AWS Lambda
Date analyse	24/06/2026

1. Rôle et contexte

La Lambda **archive_anaplan-prd** est responsable de l'archivage mensuel des fichiers CSV produits par Anaplan. Elle s'inscrit dans le pipeline Data de la plateforme ACF de BPIFrance, entre la source Anaplan et la couche Silver du Data Lake.

Son rôle précis est d'effectuer, une fois par mois, les opérations suivantes :

- Vérifier que tous les fichiers attendus de l'export mensuel Anaplan sont bien présents dans S3
- Lire la période d'export depuis un fichier de contrôle CSV produit par Anaplan
- Calculer le dernier jour du mois correspondant à cette période
- Copier chaque fichier CSV vers la couche Silver avec un partitionnement par eventdate
- Supprimer les fichiers sources après archivage
- Déclencher le Crawler Glue pour mettre à jour le catalogue de données
- Nettoyer les fichiers de contrôle en fin de traitement

Position dans le pipeline :

Anaplan (export mensuel)
→ S3 ACF Anaplan Data (Output/Export_Mensuel/)
→ Lambda archive_anaplan-prd
→ S3 ACF Silver Data (Output/output_archive_anaplan/Export_Mensuel/{dossier}/eventdate=YYYY-MM-DD/)
→ Glue Crawler output_archive_anaplan
→ Catalogue Glue / Athena / Trino

2. Configuration — Variables d'environnement

La Lambda utilise les variables d'environnement suivantes, lues à l'initialisation. Les variables **SOURCE_BUCKET** et **TARGET_BUCKET** sont obligatoires — leur absence provoque une exception immédiate.

Variable	Statut	Valeur par défaut	Description
SOURCE_BUCKET	Obligatoire	bpi-fr-{env}-acf-anaplan-data	Bucket S3 source contenant les exports Anaplan
TARGET_BUCKET	Obligatoire	bpi-fr-{env}-acf-silver-data	Bucket S3 cible — couche Silver du Data Lake
SOURCE_PREFIX	Optionnel	Output/Export_Mensuel/	Préfixe S3 des fichiers sources Anaplan
ARCHIVE_PREFIX	Optionnel	Output/output_archive_anaplan/Export_Mensuel/	Préfixe de destination dans le bucket Silver
PERIOD_FILE	Optionnel	ExporterPeriodeCalcul.csv	Fichier CSV indiquant la période d'export (ex : Mar 26)
EXTRACTION_FILE	Optionnel	ExporterDateExtractionSource.csv	Fichier de date d'extraction — supprimé en fin de traitement
EXPECTED_FILES	Optionnel	[] (liste JSON)	Liste des chemins relatifs attendus — verrou de complétude de l'export

3. Flux d'exécution détaillé

3.1 Chargement de l'environnement

La Lambda commence par charger un fichier **.env** optionnel (fonction **load_env()**). Ce fichier, s'il existe localement, permet d'injecter des variables d'environnement supplémentaires — utile pour les tests locaux. En production Lambda, les variables sont déjà dans l'environnement d'exécution AWS et ce fichier est absent, ce qui est géré silencieusement.

Ensuite, les variables d'environnement obligatoires sont lues via **get_env_or_fail()**. Si **SOURCE_BUCKET** ou **TARGET_BUCKET** est absent, une exception est levée immédiatement et le traitement s'arrête (HTTP 500).

3.2 Lecture de la liste des fichiers attendus (EXPECTED_FILES)

La variable d'environnement **EXPECTED_FILES** contient une liste JSON de chemins relatifs (par rapport au **SOURCE_PREFIX**) représentant tous les fichiers que l'export Anaplan doit avoir produit avant que l'archivage puisse commencer.

Exemple de valeur **EXPECTED_FILES** :

```
["DOSSIER_A/fichier1.csv", "DOSSIER_B/fichier2.csv", "DOSSIER_C/fichier3.csv"]
```

Si cette variable est vide ou non définie, la liste est vide et la vérification est ignorée (tous les fichiers présents sont traités).

3.3 Inventaire des objets S3 sources

La fonction **list_source_objects()** liste via pagination (par blocs de 1000 objets) tous les fichiers présents dans le bucket source sous le préfixe **SOURCE_PREFIX**. Elle retourne deux structures :

- **objects** : liste complète des clés S3 absolues (ex : Output/Export_Mensuel/DOSSIER/fichier.csv)
- **relative_keys** : ensemble des chemins relatifs au **SOURCE_PREFIX** (ex : DOSSIER/fichier.csv) — utilisé pour la comparaison avec **EXPECTED_FILES**

3.4 Vérification de la complétude de l'export (verrou qualité)

La Lambda calcule les **fichiers manquants** = **EXPECTED_FILES** – fichiers présents dans S3. Si des fichiers manquent, la Lambda retourne HTTP 200 avec le message *"Fichiers manquants"* et **s'arrête sans aucun traitement**.

Importance critique : ce verrou garantit qu'Anaplan a bien livré l'intégralité de l'export mensuel avant que l'archivage démarre. Un export partiel ne sera jamais archivé — cela évite les incohérences dans le Data Lake.

3.5 Lecture du fichier de période (ExporterPeriodeCalcul.csv)

La fonction **read_period_file()** lit le fichier **ExporterPeriodeCalcul.csv** depuis le bucket source. Ce fichier est produit par Anaplan et contient un champ **PériodeExport** indiquant le mois de l'export.

Format du fichier :

```
PériodeExport
Mar 26
```

Comportements selon l'état du fichier :

Cas	HTTP	Comportement
-----	------	--------------

Fichier absent	200	Arrêt silencieux : 'Fichier ExporterPeriodeCalcul.csv absent. Aucun traitement.'
Fichier vide	200	Arrêt silencieux : 'ExporterPeriodeCalcul.csv vide. Aucun traitement.'
Champ PeriodeExport absent dans le CSV	200	Arrêt silencieux : 'Champ PeriodeExport absent.'
Erreur lecture S3	500	Exception retournée avec le détail de l'erreur
Erreur parsing CSV	500	Exception retournée avec le détail de l'erreur
Fichier correct avec PeriodeExport	—	Traitement continue vers l'étape suivante

3.6 Calcul du dernier jour du mois (eventdate)

La fonction **compute_last_date_of_month()** transforme la période textuelle en date ISO 8601 représentant le dernier jour du mois.

Règle de parsing de la période :

- Les 3 premiers caractères = mois abrégé en anglais (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec)
- Les 2 derniers caractères = année sur 2 chiffres → 2000 + YY
- Le dernier jour du mois est calculé via **monthrange(year, month)[1]**

Exemples :

PeriodeExport	eventdate calculé
Mar 26	2026-03-31
Jan 25	2025-01-31
Feb 24	2024-02-29 (année bissextile)
Nov 25	2025-11-30

3.7 Archivage des fichiers CSV

La Lambda itère sur la liste complète des objets S3 sources et applique les filtres suivants avant d'archiver un fichier :

- Le fichier ne doit pas être **ExporterPeriodeCalcul.csv** (fichier de contrôle — exclu)
- Le fichier ne doit pas être **ExporterDateExtractionSource.csv** (fichier de contrôle — exclu)
- Le chemin relatif doit avoir au moins 2 segments (ex : DOSSIER/fichier.csv)
- L'extension du fichier doit être **.csv**

Pour chaque fichier éligible, la Lambda effectue séquentiellement :

Étape A — Construction de la clé de destination :

```
destination_key = {ARCHIVE_PREFIX}{parent_directory}/eventdate={last_date_of_month}/{filename}
```

Exemple concret :

Source : Output/Export_Mensuel/CLIENTS/clients_mars.csv

Destination : Output/output_archive_anaplan/Export_Mensuel/CLIENTS/eventdate=2026-03-31/clients_mars.csv

Étape B — Copie vers le bucket cible (Silver) :

```
s3.copy_object(Bucket=TARGET_BUCKET, CopySource={Bucket: SOURCE_BUCKET, Key: source_key}, Key: destination_key)
```

Étape C — Suppression du fichier source :

```
s3.delete_object(Bucket=SOURCE_BUCKET, Key=source_key)
```

Point de risque : si la copie S3 réussit mais que la suppression échoue (exception non catchée), le fichier reste dans le bucket source. Lors d'une prochaine exécution, il serait recopié avec potentiellement une date eventdate différente si la période a changé.

3.8 Création des placeholders de dossiers

S3 ne maintient pas de dossiers réels — un "dossier" n'existe que s'il contient au moins un objet. Après suppression de tous les fichiers CSV d'un dossier, ce dossier disparaîtrait de la vue S3.

Pour éviter cela, la Lambda crée un **objet vide (placeholder)** pour chaque dossier parent des fichiers traités :

```
s3.put_object(Bucket=SOURCE_BUCKET, Key=f"{SOURCE_PREFIX}/{folder}/", Body=b"")
```

Cela maintient la structure visible dans la console S3 et dans les outils qui listent les préfixes S3, sans occuper d'espace de stockage significatif.

3.9 Déclenchement du Crawler Glue

Si au moins un fichier CSV a été traité (**processed_files > 0**), la Lambda déclenche le Crawler Glue **output_archive_anaplan** :

```
glue.start_crawler(Name="output_archive_anaplan")
```

Ce Crawler scanne le bucket Silver sous le préfixe ARCHIVE_PREFIX, détecte les nouvelles partitions eventdate créées, et met à jour le catalogue Glue. Une fois le catalogue mis à jour, les données sont immédiatement requêtables via Athena ou Trino.

3.10 Suppression des fichiers de contrôle

En toute fin de traitement, la Lambda supprime les deux fichiers de contrôle pour signaler que le cycle mensuel est terminé :

ExporterPeriodeCalcul.csv — suppression inconditionnelle :

```
s3.delete_object(Bucket=SOURCE_BUCKET, Key=period_file_key)
```

ExporterDateExtractionSource.csv — suppression silencieuse (ignoré si absent) :

```
try: s3.delete_object(Bucket=SOURCE_BUCKET, Key=extraction_file_key)
except s3.exceptions.NoSuchKey: pass
```

La suppression de ces fichiers est le signal que l'archivage est complet. Si la Lambda est ré-exécutée le même mois, elle s'arrêtera à l'étape 3.5 car ExporterPeriodeCalcul.csv n'existe plus — comportement idempotent.

4. Schéma de flux complet

#	Étape	Action	Comportement en cas d'erreur
1	Initialisation	Chargement .env + lecture variables d'env obligatoires	Erreur → HTTP 500 si SOURCE_BUCKET ou TARGET_BUCKET absent
2	Vérif. complétude	Comparaison EXPECTED_FILES vs fichiers présents dans S3	Fichiers manquants → HTTP 200, arrêt sans traitement
3	Lecture période	Lecture ExporterPeriodeCalcul.csv → extraction champ PeriodeExport	Absent / vide → HTTP 200, arrêt. Erreur lecture → HTTP 500

4	Calcul eventdate	Parsing de la période (ex: Mar 26) → calcul dernier jour du mois (ex: 2026-03-31)	Format invalide → HTTP 500
5	Archivage CSV	Pour chaque .csv éligible : copy_object vers Silver + delete_object depuis Source	Erreur S3 → exception non catchée → HTTP 500, traitement interrompu
6	Placeholders	Création d'objets vides pour maintenir les dossiers visibles dans S3	Non bloquant
7	Crawler Glue	start_crawler(output_archive_anaplan) si au moins 1 fichier traité	Non bloquant (Glue asynchrone)
8	Suppression contrôles	Suppression ExporterPeriodeCalcul.csv + ExporterDateExtractionSource.csv	Signale la fin du cycle mensuel
9	Retour	HTTP 200 avec le nombre de fichiers traités	—

5. Structure des fichiers dans S3

5.1 Avant archivage — Bucket Source (Anaplan)

s3://bpi-fr-prd-acf-anaplan-data/

■■■ Output/Export_Mensuel/

■■■ ExporterPeriodeCalcul.csv ← fichier de contrôle (période)

■■■ ExporterDateExtractionSource.csv ← fichier de contrôle (date extraction)

■■■ CLIENTS/

■ ■■■ clients_mars_2026.csv

■■■ PRODUITS/

■ ■■■ produits_mars_2026.csv

■■■ FINANCIER/

■■■ finance_mars_2026.csv

5.2 Après archivage — Bucket Cible (Silver)

s3://bpi-fr-prd-acf-silver-data/

■■■ Output/output_archive_anaplan/Export_Mensuel/

■■■ CLIENTS/

■ ■■■ eventdate=2026-03-31/

■ ■■■ clients_mars_2026.csv

■■■ PRODUITS/

■ ■■■ eventdate=2026-03-31/

■ ■■■ produits_mars_2026.csv

■■■ FINANCIER/

■■■ eventdate=2026-03-31/

■■■ finance_mars_2026.csv

La partition **eventdate=YYYY-MM-DD** est le dernier jour du mois de la période Anaplan. Elle est compatible avec Iceberg, Athena et Trino — elle permet le data skipping lors des requêtes analytiques.

6. Gestion des erreurs

Cas d'erreur	HTTP	Catché	Comportement
SOURCE_BUCKET ou TARGET_BUCKET absent	500	Non	Exception immédiate — aucun traitement
EXPECTED_FILES JSON invalide	500	Non	ValueError — aucun traitement
Fichiers attendus manquants	200	Non	Arrêt silencieux — message explicite
ExporterPeriodeCalcul.csv absent	200	Non	Arrêt silencieux — message explicite
ExporterPeriodeCalcul.csv vide	200	Non	Arrêt silencieux — message explicite
Champ PeriodeExport absent	200	Non	Arrêt silencieux — message explicite
Format PeriodeExport invalide	500	Non	ValueError avec détail
Erreur copy_object S3	500	Non	Exception non catchée — traitement interrompu (fichiers déjà copiés ne sont pas supprimés)
Erreur delete_object S3 (fichier CSV)	500	Non	Exception non catchée — fichier reste en source
ExporterDateExtractionSource.csv absent	200	Oui	NoSuchKey catchée silencieusement
Crawler Glue déjà en cours	200	Non	Exception Glue non catchée — peut faire échouer la Lambda si le Crawler est actif

7. Points de vigilance

7.1 Erreurs S3 non catchées dans la boucle d'archivage

Les appels `copy_object` et `delete_object` dans la boucle principale ne sont pas encapsulés dans un try/except. Une erreur S3 unitaire interrompt le traitement immédiatement. Les fichiers déjà copiés mais pas encore supprimés restent en source — ils seraient retraités à la prochaine exécution.

Recommandation : encapsuler chaque paire copy/delete dans un try/except individuel avec logging, pour permettre au traitement de continuer sur les autres fichiers.

7.2 Idempotence

La Lambda est **idempotente au niveau mensuel** : une fois ExporterPeriodeCalcul.csv supprimé (étape 3.10), toute réexécution s'arrête à l'étape 3.5 sans refaire l'archivage. En revanche, si la Lambda échoue entre l'étape 3.7 et 3.10, le fichier de période existe encore et une relance retraitera tous les fichiers restants — potentiellement avec recréation de partitions déjà existantes dans Silver.

7.3 Timeout Lambda

Si l'export mensuel contient un grand nombre de fichiers CSV, la durée d'exécution peut approcher le timeout Lambda (maximum 15 minutes). Chaque fichier génère 2 appels S3 (copy + delete), plus un appel put_object par dossier. Sur un export de 100 fichiers répartis dans 10 dossiers, cela représente 210 appels S3.

7.4 Dépendance au format du champ PeriodeExport

Le parsing de la période repose sur un format strict : 3 lettres de mois anglais + espace + 2 chiffres d'année. Tout écart (mois en français, année sur 4 chiffres, séparateur différent) provoque une ValueError (HTTP 500) sans fallback.

8. Fonctions internes

Fonction	Description
load_env(file_path)	Charge optionnellement un fichier .env local dans les variables d'environnement. Utilisé pour les tests locaux.
get_env_or_fail(name)	Lit une variable d'environnement obligatoire. Lève ValueError si absente.
parse_expected_files()	Parse la variable EXPECTED_FILES (JSON list) en ensemble Python. Lève ValueError si le format est invalide.
list_source_objects(bucket, prefix)	Liste tous les objets S3 sous un préfixe via pagination. Retourne la liste des clés absolues et l'ensemble des chemins relatifs.
read_period_file(bucket, key)	Lit ExporterPeriodeCalcul.csv et extrait le champ PeriodeExport. Gère les cas d'absence et d'erreur avec retours HTTP appropriés.
compute_last_date_of_month(period)	Transforme une période textuelle (ex: Mar 26) en date ISO du dernier jour du mois (ex: 2026-03-31).
lambda_handler(event, context)	Point d'entrée principal. Orchestre toutes les étapes dans l'ordre défini. L'événement entrant n'est pas utilisé.

9. Dépendances

Bibliothèque	Type	Usage
boto3	SDK AWS	Client S3 (list, copy, delete, put) et Glue (start_crawler)
csv	Standard Python	Lecture du fichier ExporterPeriodeCalcul.csv via DictReader
json	Standard Python	Parsing de la variable EXPECTED_FILES
os	Standard Python	Lecture des variables d'environnement
datetime	Standard Python	Construction de la date eventdate au format ISO 8601
calendar	Standard Python	monthrange() pour calculer le dernier jour du mois

Synthèse

La Lambda archive_anaplan-prd en résumé :

- Déclenchement mensuel (scheduled ou event-driven)
- Verrou qualité : si l'export Anaplan est incomplet, aucun traitement
- Calcul automatique de la partition eventdate depuis la période Anaplan
- Archivage CSV : copy SOURCE_BUCKET → TARGET_BUCKET (Silver) avec structure eventdate=YYYY-MM-DD
- Suppression des sources après archivage
- Placeholders S3 pour maintenir la visibilité des dossiers
- Déclenchement du Crawler Glue pour mettre à jour le catalogue
- Suppression des fichiers de contrôle = signal de fin de cycle
- Comportement idempotent : une relance dans le même mois ne refait rien

Buckets impliqués :

Source : bpi-fr-prd-acf-anaplan-data (Output/Export_Mensuel/)

Cible : bpi-fr-prd-acf-silver-data (Output/output_archive_anaplan/Export_Mensuel/)

Crawler : output_archive_anaplan